Reducing Output Size

Case Study: Frequent Itemsets

(Part 1)

OUTLINE

1. Introduction to market-basket analysis (Part 1)
2. Algorithms for frequent itemset mining (Part 1)
   - Sequential algorithms: A-Priori.
   - Approaches to handle large inputs.
3. Controlling the output size (Part 2)
   - (Frequent) closed itemsets.
   - Maximal itemsets.
   - Top-K frequent (closed) itemsets.

What will we learn?

* Some analysis tasks require the extraction of "interesting" (e.g. overrepresented) patterns from (possibly large) datasets
* Number of candidate patterns maybe huge
* GOALS
  - Efficient extraction of patterns avoiding the enumeration of uninteresting patterns
  - control the output size by suitably defining the target patterns (e.g. excluding redundant ones)

Introduction to market-basket analysis

Market-basket analysis

DATA:

- A large set of items: e.g., products sold in a supermarket
- A large set of baskets: e.g., each basket represents what a customer bought in one visit to the supermarket

GOAL: Analyze data to extract

- Frequent itemsets: subsets of items that occur together in a (surprisingly) high number of baskets
- Association rules: correlations between subsets of items.

Popular example of association rule: customers who buy diapers and milk are likely to also buy beer

Itemsets and association rules

Dataset $T = \{t_1, t_2, \ldots, t_N\}$ of N transactions (i.e., baskets) over a set I of d items, with $t_i \subseteq I$, for $1 \leq i \leq N$.

Definition (Itemset and its support)

An itemset is a subset $X \subseteq I$ and its support w.r.t. T, denoted by $\mathrm{Supp}_T(X)$, is the *fraction* of transactions of T that contain X (i.e., $|T_X|/N$, where $T_X \subseteq T$ is be the subset of transactions that contain X).

Definition (Association rule and its support and confidence)

An association rule is a rule $r : X \to Y$, with $X, Y \subset I$, $X, Y \neq \emptyset$, and $X \cap Y = \emptyset$. Its support and confidence w.r.t. T, denoted by $\mathrm{Supp}_T(r)$ and $\mathrm{Conf}_T(r)$, respectively, are defined as

$$\begin{aligned} \mathrm{Supp}_T(r) &= \mathrm{Supp}_T(X \cup Y) \\ \mathrm{Conf}_T(r) &= \mathrm{Supp}_T(X \cup Y)/\mathrm{Supp}_T(X). \end{aligned}$$

X and Y are called, respectively, the rule's antecedent and consequent.

Obs. $\mathrm{Supp}_T(X)$, $\mathrm{Supp}_T(r)$, $\mathrm{Conf}_T(r) \in [0,1]$

Example

TID	Items
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

T

$N = 5$

Itemset $X = \{Milk, Diaper, Beer\}$

$Supp_T(X) = 2/5$

Association rule $r = \{Milk, Diaper\} \rightarrow \{Beer\}$

$Supp_T(r) = 2/5 \quad Conf_T(r) = (2/5)/(3/5) = 2/3$

Rigorous formulation of the problems

Given the dataset T of N transactions over I, and given a *support threshold* minsup $\in (0, 1]$, and a *confidence threshold* minconf $\in (0, 1]$, the following two objectives can be pursued:

1. Discover all itemsets $X \neq \emptyset$ with $\text{Supp}_T(X) \geq \text{minsup}$.
   They are called frequent itemsets w.r.t. T and minsup.

2. Discover all association rules r such that $\text{Supp}_T(r) \geq \text{minsup}$ and $\text{Conf}_T(r) \geq \text{minconf}$.

Remark: we will focus on the first problem since:

- The discovery of frequent itemsets is a first step (often computationally heavier) towards the discovery of association rules.
- Many considerations made for frequent itemsets generalize to other domains where frequent patterns are relevant.

Example (minsup = minconf = 0.5)

Dataset T	
TID	Items
1	ABC
2	AC
3	AD
4	BEF

Frequent Itemsets	
Itemset	Support
A	3/4
B	1/2
C	1/2
AC	1/2

Association Rules		
Rule	Support	Confidence
A $\rightarrow$ C	1/2	2/3
C $\rightarrow$ A	1/2	1

discovery of frequent itemsets and association rules

Applications

Association analysis is one of the most prominent data mining tasks

1. Analysis of true market baskets. Chain stores keep TBs of data about what customers buy. Frequent itemsets and association rules can help the store lay out products so to "tempt" potential buyers, or decide marketing campaigns.

2. Detection of plagiarism. Consider documents (items) and sentences (transactions). For a given transaction t, its constituent items are those documents where sentence t occurs. A frequent pair of items represent two documents that share a lot of sentences ($\Rightarrow$ possible plagiarism).

3. Analysis of biomarkers. Consider transactions associated with patients, where each transaction contains, as items, biomarkers and diseases. Association rules can help associate a particular disease to specific biomarkers.

Observations

- Support and confidence measure the interestingness of a pattern (itemset or rule), and the thresholds *minsup* and *minconf* define which patterns must be considered interesting.
- Ideally, we would like to discover patterns *unlikely to be seen in a random dataset* (hypothesis testing setting). However, what is a *random dataset*?
- The choice of minsup and minconf directly influences
  - Output size: low thresholds may yield too many patterns (possibly exponential in the input size) which become hard to exploit effectively.
  - False positive/negatives: low thresholds may yield a lot of uninteresting patterns (false positives), while high thresholds may miss some interesting patterns (false negatives).

Potential output explosion

Proposition (proof omitted)

For set I of d items:

- **Number of distinct non-empty itemsets** $= 2^d - 1$
- **Number of distinct association rules** $= 3^d - 2^{d+1} + 1$.

As a consequence:

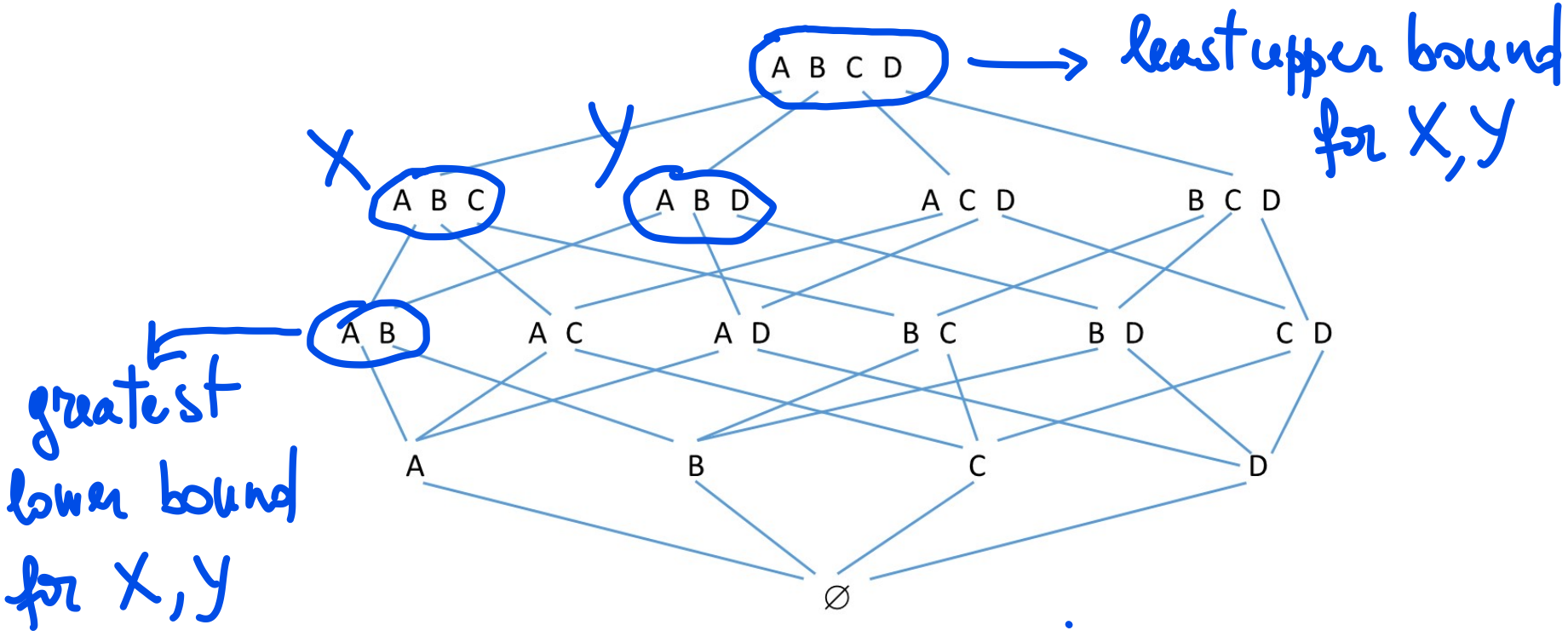
- Enumeration of all itemsets/rules (to find the interesting ones) is out of question even for small sizes (say $d > 40$)
- Complexity analysis w.r.t. input size alone might be not meaningful if output is large!

  $\Rightarrow$ We consider efficient strategies those that require *time/space polynomial in both the input and the output sizes*.

Algorithms for frequent itemset mining

Lattice of Itemsets

- The family of itemsets under $\subseteq$ forms a lattice:
  - Partially ordered set
  - For each two elements X, Y: unique least upper bound $(X \cup Y)$ and a unique greatest lower bound $(X \cap Y)$.
- The lattice can be represented through the Hasse diagram



Anti-monotonicity of support

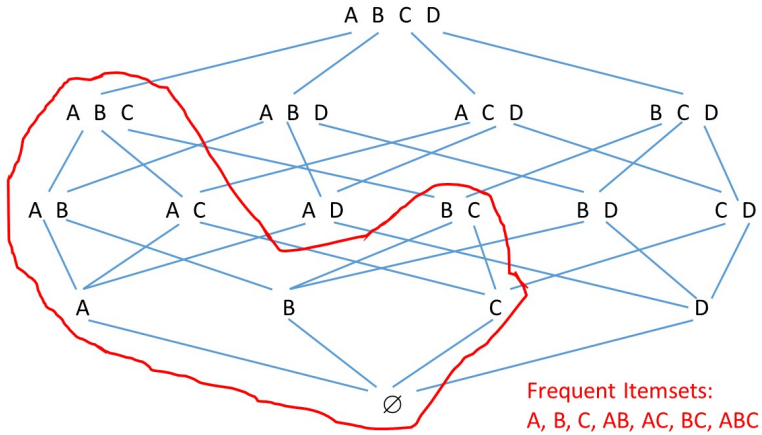
Property (Anti-monotonicity of support): For every $X, Y \subseteq I$

$$X \subseteq Y \Rightarrow \text{Supp}(X) \geq \text{Supp}(Y).$$

Important consequences: For a given support threshold

1. X is frequent $\Rightarrow$ every $W \subseteq X$ is frequent (**downward closure**)
2. X is not frequent $\Rightarrow$ every $W \supseteq X$ is not frequent

$\Rightarrow$ frequent itemsets form a sublattice closed downwards

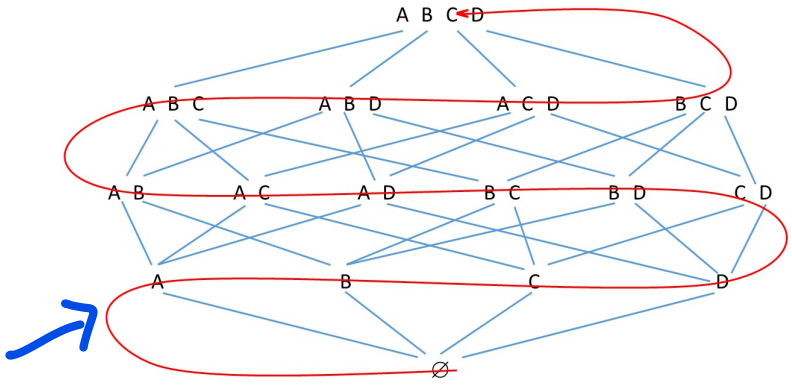


Efficient mining of Frequent Itemsets

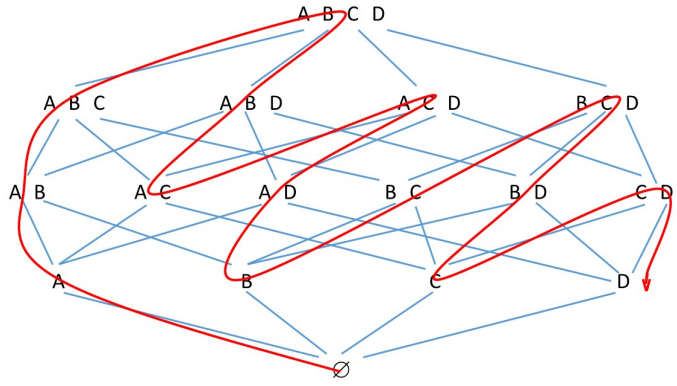
Key objective: Careful exploration of the lattice of itemsets exploiting anti-monotonicity of support

Two main approaches:

Breadth First



Depth First



A-Priori algorithm

A-Priori is a popular, paradigmatic data mining algorithm proposed by Agrawal and Srikant in 1994 at VLDB [AS94]

In 2006 it was included in the Top-10 data mining algorithms.

Uses the breadth-first approach.

A-Priori algorithm: high-level strategy

$T \equiv$ Dataset of N transactions over I, minsup

$F_k \equiv \{$Frequent itemsets w.r.t. T and minsup of lenght $k\}$

$k \geq 1$

* Compute F_1 "directly"
* For every $k > 1$
  - Use F_{k-1} to compute $C_k \supseteq F_k$ and exploit anti-monotonicity to make C_k small (candidates)
  - Compute the support of each $X \in C_k$ and return only the frequent ones (set F_k)

A-Priori algorithm: pseudocode

Input Dataset T of N transactions over I, minsup

Output $\{(X, \text{Supp}_T(X)) \ : \ X \subseteq I \wedge \text{Supp}_T(X) \geq \text{minsup}\}$

Assumptions and notation

- We assume that transactions/itemsets are represented as sorted vectors, based on some total ordering of the items.
- For every itemset $X \subseteq I$, define its absolute support as the number of transactions that contain X

$$\sigma(X) = \text{Supp}(X) \cdot N$$

A-Priori algorithm: pseudocode

$k \leftarrow 1$

Compute $F_1 = \{i \in I \ ; \ \mathtt{Supp}(\{i\}) \geq \mathtt{minsup}\}$

Compute $O_1 = \{(X, \mathtt{Supp}(X)) \ ; \ X \in F_1\}$

repeat

  $k \leftarrow k + 1$

  $C_k \leftarrow$ APRIORI-GEN(F_{k-1}) /* *Candidates* */

  for each $c \in C_k$ **do** $\sigma(c) \leftarrow 0$

  for each $t \in T$ **do**

    for each $c \in C_k$ **do**

      if $c \subseteq t$ **then** $\sigma(c) \leftarrow \sigma(c) + 1$

  $F_k \leftarrow \{c \in C_k \ : \ \sigma(c) \geq N \cdot \mathtt{minsup}\}$;

  $O_k \leftarrow \{(X, \sigma(X)/N) \ ; \ X \in F_k\}$

until $F_k = \emptyset$

return $\bigcup_{k \geq 1} O_k$

Computation of F_1 can be done easily doing a scan of T and using a Map to store the items and their occurrences in T

We can safely stop at this point since by anti-monotonicity there cannot exist frequent itemsets of length $> k$

APRIORI-GEN(F): high-level strategy

2 PHASES → e.g. $F = F_{k-1}$

* CANDIDATE GENERATION: merge pairs $X, Y \in F$ which share all but the last item

  e.g. $X = ABCD$ $Y = ABCE \Rightarrow X \cup Y = ABCDE$

* CANDIDATE PRUNING: remove a candidate if it contains one subset not in F

  e.g. if $ABCDE$ is such that $ACDE \notin F \Rightarrow ABCDE$ is removed

$\Rightarrow$ Pruning is done "a priori" without computing supports, but only exploiting anti-monotonicity!

APRIORI-GEN(F): pseudocode

```
Let ℓ − 1 be the size of each itemset in F
Φ ← ∅
/* Candidate Generation */
for each X, Y ∈ F s.t. X ≠ Y ∧ X[1...ℓ − 2] = Y[1...ℓ − 2] do
   add X ∪ Y to Φ
/* Candidate Pruning */
for each Z ∈ Φ do
   for each Y ⊂ Z s.t. |Y| = ℓ − 1 do
      if (Y ∉ F) then {remove Z from Φ; exit inner loop}
return Φ
```

Obs. Φ contains itemsets of length ℓ

Observation: no itemset is generated twice.

Example

DATASET T	
TID	ITEMS
1	ACD
2	BEF
3	ABCEF
4	ABF

- $N = 4$ **transactions**, $d = 6$ **items**.
- Let's fix $\text{minsup} = 0.5$.
- **Observation:** a brute force strategy would compute the support of $2^d - 1 = 63$ itemsets.

Example

DATASET T	
TID	ITEMS
1	ACD
2	BEF
3	ABCEF
4	ABF

ITEMSET	SUPPORT
A	3/4
B	3/4
C	1/2
~~D~~	~~1/4~~
E	1/2
F	3/4

6

D is not frequent

Set F_1	
ITEMSET	SUPPORT
A	3/4
B	3/4
C	1/2
E	1/2
F	3/4

Example

DATASET T	
TID	ITEMS
1	ACD
2	BEF
3	ABCEF
4	ABF

Set C_2		
ITEMSET		SUPPORT
After Generation	After Pruning	
AB	AB	1/2
AC	AC	1/2
AE	AE	1/4
AF	AF	1/2
BC	BC	1/4
BE	BE	1/2
BF	BF	3/4
CE	CE	1/4
CF	CF	1/4
EF	EF	1/2

10

Example

Set F_2	
ITEMSET	SUPPORT
AB	1/2
AC	1/2
AF	1/2
BE	1/2
BF	3/4
EF	1/2

DATASET T	
TID	ITEMS
1	ACD
2	BEF
3	ABCEF
4	ABF

Set C_3		
ITEMSET		SUPPORT
After Generation	After Pruning	
ABC		
ABF	ABF	1/2
ACF		
BEF	BEF	1/2

2

Example

Set F_3	
ITEMSET	SUPPORT
ABF	1/2
BEF	1/2

$C_4 = \phi \Rightarrow F_4 = \phi \Rightarrow$ STOP

Observations

* ABF and BEF share 2 items but for sure ABF ∪ BEF = ABEF must be frequent because if it were frequent then ABE and ABF would also be frequent ⇒ ABEF would be generated by APRIORI-GEN
* Altogether the algorithm computes the support of: 6 + 10 + 2 = 18 itemsets out of the pool of 63 total non-empty subsets of items

Correctness of A-Priori

Theorem (Correctness)

The A-Priori algorithm for mining frequent itemsets is correct

It is sufficient to show that for each $k \geq 1$ the set F_k computed by the algorithm is EXACTLY the set of frequent itemsets of length k.

We argue by induction on $k \geq 1$

Proof of Theorem

Basis $k=1$ trivial

Induction step:

Hp. F_{k-1} computed by A-Priori is the set of all frequent itemsets of length $k-1$

$C_k = \text{APRIORI-GEN}(F_{k-1})$

It is sufficient to show that C_k contains all frequent itemsets of length k

Proof of Theorem

Let X be an arbitrary frequent itemset of length k

$$X = X[1]\,X[2]\,\ldots\,X[k]$$

Define

$$X_1 = X[1]\,X[2]\,\ldots\,X[k-2]\,X[k-1]$$

$$X_2 = X[1]\,X[2]\,\ldots\,X[k-2]\,X[k]$$

E.g. $X = ABCD$ $\quad X_1 = ABC$ $\quad X_2 = ABD$

$k = 4$ Obs. $\mathrm{Supp}_T(X_i) \geq \mathrm{Supp}_T(X)$

by anti-monotonicity

Proof of Theorem

Clearly $X = X_1 \cup X_2$ (by construction)

Also, X_1, X_2 are frequent (by anti-monotonicity)

$\Rightarrow X_1, X_2 \in F_{k-1}$ (by inductive Hp)

$\Rightarrow X_1 \cup X_2 = X$ must be added by APRIORI-GEN to C_k in the Candidate Generation phase, and cannot be removed in the Candidate Pruning phase since all of its subsets are frequent, by antimonotonicity. □

Efficiency of A-Priori

A-Priori owes its popularity to its efficient performance especially for sparse datasets (i.e., few frequent itemsets).

The efficiency of A-Priori is due to the following features:

- A few passes over the dataset (typically very large):

  $k_{\max} + 1$ passes, where $k_{\max}$ is the max length of a frequent itemset. Note that for a sparse dataset $k_{\max}$ is small.

- Support is computed only for a few non-frequent itemsets:

  (see lemma in next slide) this is due to candidate generation and pruning which exploit antimonotonicity of support.

- Several optimizations are known for computing candidates supports (typically the most time-consuming task).

Efficiency of A-Priori

Lemma

For dataset T over d items and a threshold minsup, let M be the number of frequent itemsets returned by A-Priori. Then,

The algorithm computes the support for $\leq d + \min\{M^2, dM\}$ itemsets.

A-Priori computes the support for

* the d items
* the itemsets in C_k $\forall\ k \geq 2$

$\Rightarrow$ We just have to prove an upper bound to $|C_k|$

Proof of Lemma

m_k = # of frequent itemsets of length k

$k \geq 1$

$$\Rightarrow M = \sum_{k \geq 1} m_k$$

Now we show that $|C_k| \leq \min\{(m_{k-1})^2, d m_{k-1}\}$ for any $k \geq 2$

Observe that $\forall Z \in C_k \; \exists X, Y$ frequent itemsets of length $k-1$ such that

Proof of Lemma

* $Z = X \cup Y$
* $Z = Xa$ where $a \equiv$ last item of Y

$\Rightarrow$

* $|C_k| \leq \binom{m_{k-1}}{2} \leq (m_{k-1})^2$
* $|C_k| \leq d \cdot m_{k-1}$

$\Rightarrow |C_k| \leq \min\{(m_{k-1})^2, d \cdot m_{k-1}\}$

Hence, A-Priori computes supports for

Proof of Lemma

$$d + \sum_{k \geq 2} |C_k| \quad \text{itemsets}$$

$$\leq d + \sum_{k \geq 2} \min\left\{ (m_{k-1})^2, d\, m_{k-1} \right\}$$

$$\leq d + \min\left\{ \sum_{k \geq 2} (m_{k-1})^2, d \sum_{k \geq 2} m_{k-1} \right\}$$

$$\leq d + \min\left\{ \left(\sum_{k \geq 2} m_{k-1} \right)^2, d \sum_{k \geq 2} m_{k-1} \right\}$$

$$= d + \min\left\{ M^2, dM \right\} \qquad \square$$

Efficiency A-Priori

The following theorem is an easy consequence of the lemma.

Theorem

The A-Priori algorithm for mining frequent itemsets can be implemented in time polynomial in both the input size (sum of all transaction lengths) and the output size (sum of all frequent itemsets legnths).

A-Priori in practice

The most time-consuming step is the counting of supports of candidates (sets $C_2, C_3, \ldots$)

Main performance issues:

- A-Priori requires a pass over the entire dataset checking each candidate against each transaction (*a lot for large input/output sizes!*)
- The number of candidates may still be very large stressing storage and computing capacity.

  Obs: the issue may become critical for C_2, which contains *all pairs of frequent items* (e.g., suppose 1M items!). As k grows larger, the cardinality of F_{k-1}, hence of C_k, drops.

Many efforts in the last 2 decades to address the above issues!

Other approaches to frequent itemsets mining

Several algorithms based on depth-first mining strategies have been devised and tested (large body of literature!)

Their main features are:

- avoiding several passes over the entire dataset of transactions.
- confining the support counting of longer itemsets to small projections of the dataset.

Prominent examples:

- FP-Growth: proposed in [HJY00] is a popular algorithm for mining frequent itemsets. It is implemented in Spark!
- Patricimine: an improvement to FP-Growth (*devised in Padova!* [PZ03]) is one of the fastest algorithms to date

Exercises

Exercise

Let T be a dataset of transaction. For an itemset $X = \{x_1, x_2, \ldots, x_k\}$, define the measure:

$$\zeta(X) = \min\{\text{Conf}(x_i \to X - \{x_i\}) \ : \ 1 \leq i \leq k\}.$$

Say whether ζ is *monotone*, *anti-monotone* or neither one. Justify your answer.

Exercises

Exercise

Let T be a dataset of N *strings* over an alphabet Σ. Define the support of a string X with respect to T as:

$$\mathrm{Supp}_T(X) = \frac{|\{t \in T \ : \ X \text{ is a substring of } t\}|}{N},$$

that is, the fraction of strings of T that contain X as a substring. (We say that X *is a substring of* t *if it occurs in contiguous positions of* t.)

1. Identify a suitable anti-monotonicity property for $\mathrm{Supp}_T(\cdot)$.
2. Given a support threshold minsup $\in (0, 1)$ let F_k be the set of strings of length k of support $\geq$ minsup. Define

   $$C_{k+1} = \{Xa \ ; \ X \in F_k, a \in \Sigma, \text{ and } X[1] \cdots X[k-1]a \in F_k\}.$$

   Show that $F_{k+1} \subseteq C_{k+1}$.

Approaches to handle large inputs

Suppose that we must compute the frequent itemsets w.r.t. a very large dataset T and a threshold minsup. Two approaches can be used:

1. **Partition-based approach:**
   - Partition T into ℓ subsets $T_1, T_2, \ldots, T_\ell$
   - For each i separately, determine the set F^{T_i} of frequent itemsets w.r.t. T_i and minsup.
   - Compute the support (w.r.t. T) for the itemsets in $\cup_{i=1,\ell} F^{T_i}$, returning those of support $\geq$ minsup.
2. **Sampling approach:** compute the frequent itemsets from a small sample of T.

Remarks:

- The partitioning approach computes the exact output but may be inefficient.
- The sampling approach computes an approximation but is more efficient.

Partition-based approach

The partition-based approach is simple to implement in MapReduce, and the following exercise shows why it is correct.

Exercise

Let T be a dataset of transactions, partitioned into $T_1, T_2, \ldots, T_\ell$. For a given support threshold minsup, define

- F^T = the set of frequent itemsets w.r.t. T,
- F^{T_i} = the set of frequent itemsets w.r.t. T_i,

Show that $F^T \subseteq \bigcup_{i=1}^{\ell} F^{T_i}$.

Remark. The problem with the partition-based approach is that the itemsets which are frequent in the individual partitions (F^{T_i}'s) may be many more than those that are frequent w.r.t. T (F^T), and computing the supports of all of them may be time consuming.

Sampling-based approach

Do we really need to process the entire dataset?

No, if we are happy with some

approximate set of frequent itemsets

(but quality of approximation under control)

Sampling-based approach

Definition (Approximate frequent itemsets)

Consider a dataset T of transactions over the set of items I, a support threshold $\text{minsup} \in (0, 1]$, and a parameter $\epsilon > 0$. A set C of pairs (X, s_X), with $X \subseteq I$ and $s_x \in (0, 1]$, is an ϵ-approximation of the set $F_{T,\text{minsup}}$ of frequent itemsets and their supports if the following conditions are satisfied:

1. For each $X \in F_{T,\text{minsup}}$ there exists a pair $(X, s_X) \in C$
2. For each $(X, s_X) \in C$,
   - $\text{Supp}_T(X) \geq \text{minsup} - \epsilon$
   - $|\text{Supp}_T(X) - s_X| \leq \epsilon$.

Observations

* Condition 1 ensures that C does not miss truly frequent itemsets (i.e. there are no false negatives)
* Condition 2 ensures that
  - C contains no itemsets of very low support
  - s_X is a good estimate of the true support $Supp_T(X)$

Sampling-based approach: algorithm

Consider a dataset T of N transactions over I and a support threshold $\text{minsup} \in (0, 1]$. The following general algorithm can be used to compute an approximation to the frequent itemsets:

- Fix a suitably lower threshold $f(\text{minsup}) < \text{minsup}$.
- Draw a random sample $S \subseteq T$ with uniform probability and with replacement.
- Return the set of pairs

$$C = \left\{ (X, s_x = \text{Supp}_S(X)) \ : \ X \in F_{S, f(\text{minsup})} \right\},$$

where $F_{S,f(\text{minsup})}$ are the frequent itemsets w.r.t. S and minsup.

How well does C approximate the true frequent itemsets and their supports?

Sampling-based approach

Theorem (Riondato-Upfal)

Let h be the maximum transaction length and let ϵ, δ be suitable design parameters in $(0, 1)$. There is a constant $c > 0$ such that if

$$f(minsup) = minsup - \frac{\epsilon}{2} \quad AND \quad |S| = \frac{4c}{\epsilon^2}\left(h + \log\frac{1}{\delta}\right)$$

then with probability at least $1 - \delta$ the set C returned by the algorithm is an ϵ-approximation of the set of frequent itemsets and their supports.

Terminology: to account for the probabilistic nature of the result in the theorem, the algorithm is said to provide in this case an (ϵ, δ)-approximation to the frequent itemset mining problem.

Observations

* Sample size $|S|$ does not depend on $N = |T|$ and does not depend on minsup
  Only a week dependence on T through h

* The algorithm can be implemented in MapReduce in $O(1)$ rounds assuming $h = o(N)$ and that $|C|$ is also $o(N)$

Proof of the theorem

The result of the theorem relies on the following lemma which can be proved through an elegant exploitation of the notion of VC-dimension (see [RU14] for details).

Lemma

Using the sample size $|S|$ specified in the theorem, the algorithm ensures that with probability $\geq 1 - \delta$ for each itemset X it holds that

$$|Supp_T(X) - Supp_S(X)| \leq \epsilon/2.$$

We omit the proof of the lemma

Proof of the theorem

We need to show

output of the algorithm

① if $X \in F_{T,minsup} \Rightarrow \exists (X, s_X)$ in C

if $X \in F_{T,minsup}$ then $Supp_T(X) \geq minsup$

$\Rightarrow Supp_S(X) \geq Supp_T(X) - \varepsilon/2$ (by lemma)

$\geq minsup - \varepsilon/2$

$= f(minsup)$

$\Rightarrow (X, s_X = Supp_S(X))$ is returned in output within the set C

Proof of the theorem

② * If $(X, s_X) \in C$ then $\mathrm{supp}_T(X) \geq \mathrm{minsup} - \varepsilon$

It is true since

$$\mathrm{supp}_T(X) \geq \mathrm{supp}_S(X) - \varepsilon/2 \quad \text{(by lemma)}$$

$$= s_X - \varepsilon/2$$

$$\geq f(\mathrm{minsup}) - \varepsilon/2$$

$$= \mathrm{minsup} - \varepsilon/2 - \varepsilon/2 = \mathrm{minsup} - \varepsilon$$

* $|\mathrm{supp}_T(X) - s_X| < \varepsilon$

By Lemma $|\mathrm{supp}_T(X) - s_X = \mathrm{supp}_S(X)| \leq \varepsilon/2 < \varepsilon$

□

References

LRU14 J.Leskovec, A.Rajaraman and J.Ullman. Mining Massive Datasets. Cambridge University Press, 2014. Chapter 6.

AS94 R.Agrawal and R.Srikant. Fast algorithms for mining association rules. Proc. 20th International Conference on Very Large Data Bases (VLDB) 1994.

HPY00 J.Han, J.Pei, Y.Yin. Mining frequent patterns without candidate generation. Proc. SIGMOD Conference, 2000.

PZ03 A.Pietracaprina, D.Zandolin: Mining Frequent Itemsets using Patricia Tries. Proc. FIMI 2003.

RU14 M. Riondato, E. Upfal: Efficient Discovery of Association Rules and Frequent Itemsets through Sampling with Tight Performance Guarantees. ACM Trans. on Knowledge Discovery from Data, 2014.